C# / .NET Learning Roadmap

Topic Notes — Section 15: Authentication and Authorization

Topics: 15.1 through 15.8

Date: 29 June 2026

Section 15 — Authentication and Authorization

This section covers eight topics taught across the auth domain: 15.1 (JWT Authentication), 15.2 (ASP.NET Core Identity), 15.3 (Role-based and Policy-based Authorization), 15.4 (HTTPS, CORS, and Security Headers), 15.5 (Secrets Management), 15.6 (Input Validation and SQL Injection Prevention), 15.7 (Data Protection API and Encryption), 15.8 (Audit Logging and Sensitive Data Handling).

15.1. JWT Authentication (JSON Web Tokens)

Builds on: 12.2 (DI -- auth services on builder.Services), 12.6 (middleware -- UseAuthentication/UseAuthorization in pipeline), 13.2 (controllers -- [Authorize] attribute).

What is JWT and why stateless auth?

```
Traditional session:          JWT:
  Login -> session in DB        Login -> signed token issued
  Cookie returned               Token returned
  Every request: DB lookup      Every request: validate signature
  Stateful -- hard to scale     Stateless -- scales easily
```

JWT structure

```
// Three base64 parts separated by dots:
Header.Payload.Signature

// Header:  { "alg": "HS256", "typ": "JWT" }
// Payload (claims):
// {
//   "sub":   "123",
//   "email": "s@zoho.com",
//   "role":  "Admin",
//   "exp":   1716234000
// }
// Signature: HMACSHA256(header+payload, secret)
// Tamper with payload -> signature fails -> rejected
```

Configuring JWT in Program.cs

```
// appsettings.json:
// { "JwtSettings": {
//     "Secret":        "32-char-min-secret-key",
//     "Issuer":        "MyApp.Api",
//     "Audience":      "MyApp.Client",
//     "ExpiryMinutes": 60 } }

builder.Services
    .AddAuthentication(options =>
    {
        options.DefaultAuthenticateScheme =
            JwtBearerDefaults.AuthenticationScheme;
        options.DefaultChallengeScheme =
```

```
            JwtBearerDefaults.AuthenticationScheme;
    })
    .AddJwtBearer(options =>
    {
        var s = builder.Configuration
            .GetSection("JwtSettings");
        options.TokenValidationParameters =
            new TokenValidationParameters
            {
                ValidateIssuer           = true,
                ValidateAudience         = true,
                ValidateLifetime         = true,
                ValidateIssuerSigningKey = true,
                ValidIssuer   = s["Issuer"],
                ValidAudience = s["Audience"],
                IssuerSigningKey =
                    new SymmetricSecurityKey(
                        Encoding.UTF8.GetBytes(
                            s["Secret"]!))
            };
    });

// Middleware -- order matters:
app.UseAuthentication(); // who are you?
app.UseAuthorization();  // what can you do?
```

Generating a JWT token at login

```
public string GenerateToken(
    int userId, string email, string role)
{
    var s = _config.GetSection("JwtSettings");
    var claims = new[]
    {
        new Claim(
            JwtRegisteredClaimNames.Sub,
            userId.ToString()),
        new Claim(
            JwtRegisteredClaimNames.Email, email),
        new Claim(ClaimTypes.Role, role),
        new Claim(
            JwtRegisteredClaimNames.Jti,
            Guid.NewGuid().ToString())
    };

    var key = new SymmetricSecurityKey(
        Encoding.UTF8.GetBytes(s["Secret"]!));
    var creds = new SigningCredentials(
        key, SecurityAlgorithms.HmacSha256);

    var token = new JwtSecurityToken(
        issuer:   s["Issuer"],
        audience: s["Audience"],
        claims:   claims,
        expires:  DateTime.UtcNow.AddMinutes(
```

```
            int.Parse(s["ExpiryMinutes"]!)),
        signingCredentials: creds);

    return new JwtSecurityTokenHandler()
        .WriteToken(token);
}
```

Protecting endpoints and reading claims

```
[Authorize]                          // any auth user
[Authorize(Roles = "Admin")]         // role required
[Authorize(Roles = "Admin,Manager")] // OR logic
[AllowAnonymous]                     // bypass [Authorize]

// Read claims in controller:
var userId = User.FindFirst(
    JwtRegisteredClaimNames.Sub)?.Value;

// Refresh tokens -- short-lived access + long-lived
// refresh token stored in DB:
// POST /auth/refresh -> validate refresh token
//   -> rotate (revoke old, issue new)
//   -> return new access + refresh token
```

Key Takeaways for 15.1

- JWT = signed token with claims; server validates signature -- no DB lookup
- Three parts: Header, Payload (claims), Signature
- AddAuthentication().AddJwtBearer() + TokenValidationParameters
- UseAuthentication() before UseAuthorization() -- order matters
- [Authorize] / [Authorize(Roles)] / [AllowAnonymous] on controllers/actions
- Refresh tokens: short access token (60 min) + long refresh token (stored in DB)

Debugging Tips

Symptom	Cause	Fix
401 on every request despite valid token	UseAuthentication() not called or after UseAuthorization()	Ensure UseAuthentication() is before UseAuthorization()
IDX10503: Signature validation failed	Wrong secret key or encoding mismatch	Use same Encoding.UTF8.GetBytes(secret) for signing and validation
Token accepted but claims empty	Claims not added when generating token	Add new Claim(JwtRegisteredClaimNames.Sub, userId) in claims array
Token expires too quickly	ExpiryMinutes too short or server clock skew	Increase expiry; set ClockSkew = TimeSpan.Zero in TokenValidationParameters

15.2. ASP.NET Core Identity

Builds on: 15.1 (JWT -- Identity is the user store JWT tokens are issued against), 14.2 (DbContext -- Identity adds tables via EF Core), 12.2 (DI -- Identity registered on builder.Services).

Setting up Identity

```
// dotnet add package
//   Microsoft.AspNetCore.Identity.EntityFrameworkCore

// Extend ApplicationUser:
public class ApplicationUser : IdentityUser<int>
{
    public string FirstName { get; set; }
        = string.Empty;
    public string LastName { get; set; }
        = string.Empty;
    public DateTime CreatedAt { get; set; }
        = DateTime.UtcNow;
}

// Identity-aware DbContext:
public class AppDbContext
    : IdentityDbContext<
        ApplicationUser, IdentityRole<int>, int>
{
    public DbSet<Order> Orders { get; set; }

    public AppDbContext(
        DbContextOptions<AppDbContext> options)
        : base(options) { }

    protected override void OnModelCreating(
        ModelBuilder m)
    {
        base.OnModelCreating(m); // required!
        m.ApplyConfigurationsFromAssembly(
            typeof(AppDbContext).Assembly);
    }
}
```

Registering Identity in Program.cs

```
builder.Services
    .AddIdentity<ApplicationUser,
                 IdentityRole<int>>(options =>
    {
        options.Password.RequiredLength       = 8;
        options.Password.RequireDigit         = true;
        options.Password.RequireUppercase     = true;
        options.Lockout.MaxFailedAccessAttempts = 5;
        options.Lockout.DefaultLockoutTimeSpan =
            TimeSpan.FromMinutes(15);
        options.User.RequireUniqueEmail       = true;
    })
```

```
    .AddEntityFrameworkStores<AppDbContext>()
    .AddDefaultTokenProviders();
```

UserManager -- register and login

```
// Register:
var user = new ApplicationUser
{
    UserName  = dto.Email,
    Email     = dto.Email,
    FirstName = dto.FirstName
};
// CreateAsync hashes password automatically:
var result = await _userManager
    .CreateAsync(user, dto.Password);

if (!result.Succeeded)
    return BadRequest(result.Errors
        .Select(e => e.Description));

await _userManager
    .AddToRoleAsync(user, "User");

// Login:
var user = await _userManager
    .FindByEmailAsync(dto.Email);

var valid = await _userManager
    .CheckPasswordAsync(user, dto.Password);

if (!valid)
{
    await _userManager.AccessFailedAsync(user);
    return Unauthorized(
        "Invalid email or password");
}

if (await _userManager.IsLockedOutAsync(user))
    return Unauthorized(
        "Account locked. Try again later.");

await _userManager
    .ResetAccessFailedCountAsync(user);

var roles = await _userManager
    .GetRolesAsync(user);
var token = _tokenService.GenerateToken(
    user.Id, user.Email!,
    roles.FirstOrDefault() ?? "User");
```

Role management and seeding

```
// Seed roles at startup:
using (var scope = app.Services.CreateScope())
{
```

```
    var roleManager = scope.ServiceProvider
        .GetRequiredService<
            RoleManager<IdentityRole<int>>>();

    foreach (var role in
        new[] { "Admin", "Manager", "User" })
    {
        if (!await roleManager
                .RoleExistsAsync(role))
            await roleManager.CreateAsync(
                new IdentityRole<int>(role));
    }
}

// Manage roles:
await _userManager
    .AddToRoleAsync(user, "Admin");
await _userManager
    .RemoveFromRoleAsync(user, "Admin");
var roles = await _userManager
    .GetRolesAsync(user);

// Custom claims stored in DB:
await _userManager.AddClaimAsync(
    user,
    new Claim("department", "Engineering"));
```

Password reset flow

```
// Step 1 -- generate token:
var token = await _userManager
    .GeneratePasswordResetTokenAsync(user);
// Send by email; token is time-limited

// Step 2 -- reset:
var result = await _userManager
    .ResetPasswordAsync(
        user, dto.Token, dto.NewPassword);

// Identity tables created by migrations:
// AspNetUsers, AspNetRoles, AspNetUserRoles,
// AspNetUserClaims, AspNetRoleClaims,
// AspNetUserLogins, AspNetUserTokens
```

Key Takeaways for 15.2

- Identity manages user store -- users, hashed passwords, roles, claims in DB
- UserManager<T> -- create user, check password, manage roles/claims
- IdentityDbContext -- base class that adds Identity tables via migrations
- CreateAsync(user, password) -- hashes password automatically
- CheckPasswordAsync() -- validates hash; combine with lockout tracking
- AddDefaultTokenProviders() -- required for password reset and email confirmation

Debugging Tips

Symptom	Cause	Fix
CreateAsync always fails with password errors	Password rules not met by the test password	Check options.Password.* settings; relax rules in Development
FindByEmailAsync returns null despite user existing	UserName and Email not both set to email value	Set both UserName = email AND Email = email on user object
Migration fails after adding Identity	OnModelCreating does not call base.OnModelCreating	Add base.OnModelCreating(modelBuilder) before your own config
Roles not found at startup seeding	Role seeding not awaited or runs before DI ready	Use using var scope = app.Services.CreateScope(); await all ops

15.3. Role-Based and Policy-Based Authorization

Builds on: 15.1 (JWT -- roles and claims from token), 15.2 (Identity -- roles stored in user store), 12.2 (DI -- policies registered on builder.Services).

Role-based authorization

```
[Authorize(Roles = "Admin")]         // single role
[Authorize(Roles = "Admin,Manager")] // OR -- any one

// AND -- must satisfy both:
[Authorize(Roles = "Admin")]
[Authorize(Roles = "Manager")]
public IActionResult AdminAndManager() { }

// Check in code:
if (User.IsInRole("Admin")) { ... }
```

Policy-based authorization

```
// Program.cs:
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("AdminOnly", p =>
        p.RequireRole("Admin"));

    options.AddPolicy("VerifiedUser", p =>
        p.RequireClaim(
            "email_verified", "true"));

    options.AddPolicy("PremiumMember", p =>
        p.RequireAuthenticatedUser()
         .RequireClaim(
             "subscription", "Premium"));

    options.AddPolicy("OrderManager", p =>
        p.RequireRole("Manager", "Admin")
```

```
          .RequireClaim(
              "department", "Operations"));

    options.AddPolicy("Over18", p =>
        p.RequireAssertion(ctx =>
            ctx.User.HasClaim(c =>
                c.Type == "age" &&
                int.TryParse(c.Value,
                    out var age) &&
                age >= 18)));
});

// Apply:
[Authorize(Policy = "AdminOnly")]
[HttpDelete("{id:int}")]
public async Task<IActionResult>
    Delete(int id) { }
```

Custom requirement and handler

```
// Requirement:
public class OrderOwnerRequirement
    : IAuthorizationRequirement { }

// Handler:
public class OrderOwnerHandler
    : AuthorizationHandler<
        OrderOwnerRequirement, Order>
{
    protected override Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        OrderOwnerRequirement req,
        Order resource)
    {
        var uid = context.User
            .FindFirst(
                JwtRegisteredClaimNames.Sub)
            ?.Value;

        if (int.TryParse(uid, out var userId)
            && resource.UserId == userId)
            context.Succeed(req);

        if (context.User.IsInRole("Admin"))
            context.Succeed(req);

        return Task.CompletedTask;
    }
}

// Register:
builder.Services.AddSingleton<
    IAuthorizationHandler, OrderOwnerHandler>();

// Resource-based check in controller:
```

```
var result = await _authService.AuthorizeAsync(
    User, order, "OrderOwner");
if (!result.Succeeded) return Forbid();
```

Key Takeaways for 15.3

- Role-based: [Authorize(Roles="Admin")] -- simple, coarse access control
- Policy-based: [Authorize(Policy="Name")] -- flexible, combines roles + claims
- RequireRole(), RequireClaim(), RequireAssertion() -- built-in policy methods
- Custom IAuthorizationRequirement + AuthorizationHandler<T> for complex rules
- Resource-based: inject IAuthorizationService, call AuthorizeAsync(user, resource, policy)
- Multiple [Authorize] = AND logic; comma-separated roles = OR logic

```
Method                      | Effect
----------------------------+---------------------------
RequireAuthenticatedUser() | User must be logged in
RequireRole("Admin")       | User must have role
RequireClaim("t", "v")     | User must have claim
RequireAssertion(ctx => ..)| Custom C# condition
```

Debugging Tips

Symptom	Cause	Fix
403 despite correct role	Role claim key mismatch -- ClaimTypes.Role vs role	Set options.TokenValidationParameters .RoleClaimType = ClaimTypes.Role
Policy always fails even with claim present	Claim value case mismatch -- True vs true	Claims are case-sensitive; ensure issuer and policy use identical values
AuthorizeAsync always returns failed	Handler not registered in DI	Add builder.Services.AddSingleton<IAut horizationHandler, MyHandler>()
[Authorize] on controller bypassed	Action has [AllowAnonymous] override	Remove [AllowAnonymous] if action should be protected

15.4. HTTPS, CORS, and Security Headers

Builds on: 12.6 (middleware -- security features are middleware), 13.1 (routing -- CORS applies at routing level).

HTTPS -- enforcing encrypted connections

```
// Only in Production:
if (!app.Environment.IsDevelopment())
    app.UseHsts();

app.UseHttpsRedirection(); // HTTP -> HTTPS

// Custom HSTS:
builder.Services.AddHsts(options =>
{
    options.MaxAge = TimeSpan.FromDays(365);
```

```
    options.IncludeSubdomains = true;
    options.Preload = true;
});
```

CORS -- Cross-Origin Resource Sharing

CORS is a browser security feature only -- server-to-server calls are not affected. Define named CORS policies:

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("ProductionPolicy", p =>
        p.WithOrigins(
             "https://myapp.com",
             "https://www.myapp.com")
         .WithMethods(
             "GET","POST","PUT",
             "DELETE","PATCH")
         .WithHeaders(
             "Content-Type","Authorization")
         .AllowCredentials());

    options.AddPolicy("DevPolicy", p =>
        p.AllowAnyOrigin()
         .AllowAnyMethod()
         .AllowAnyHeader());
    // AllowAnyOrigin + AllowCredentials is invalid
});

if (app.Environment.IsDevelopment())
    app.UseCors("DevPolicy");
else
    app.UseCors("ProductionPolicy");
```

Security headers middleware

```
app.Use(async (context, next) =>
{
    // Prevent clickjacking:
    context.Response.Headers
        .Append("X-Frame-Options", "DENY");

    // Stop MIME type sniffing:
    context.Response.Headers
        .Append(
            "X-Content-Type-Options",
            "nosniff");

    // Referrer control:
    context.Response.Headers
        .Append("Referrer-Policy",
            "strict-origin-when-cross-origin");

    // Content Security Policy (XSS defence):
    context.Response.Headers
```

```
        .Append("Content-Security-Policy",
            "default-src 'self'; " +
            "script-src 'self'; " +
            "style-src 'self' 'unsafe-inline'; "+
            "img-src 'self' data: https:");

    await next();
});
```

Correct middleware order

```
if (!app.Environment.IsDevelopment())
    app.UseHsts();         // 1. HSTS
app.UseHttpsRedirection(); // 2. HTTP->HTTPS
app.Use(...);              // 3. Security headers
app.UseRateLimiter();      // 4. Rate limiting
app.UseCors(...);          // 5. CORS
app.UseAuthentication();   // 6. Who are you?
app.UseAuthorization();    // 7. What can you do?
app.MapControllers();      // 8. Route to action
```

Key Takeaways for 15.4

- UseHttpsRedirection() -- redirects HTTP to HTTPS
- UseHsts() -- tells browser to always use HTTPS (Production only)
- CORS is browser-only -- server-to-server calls unaffected
- AllowAnyOrigin() cannot be combined with AllowCredentials()
- Security headers defend against XSS and clickjacking
- CSP is the primary XSS defence -- restrict which scripts/styles load

Debugging Tips

Symptom	Cause	Fix
Browser CORS error despite policy	UseCors() registered after MapControllers()	Move UseCors() before MapControllers()
AllowCredentials throws at startup	AllowAnyOrigin() combined with AllowCredentials()	Use WithOrigins(specific URLs) when using credentials
HSTS breaking local development	UseHsts() applied in Development	Wrap in if (!app.Environment.IsDevelopment())
Security headers not in response	Header middleware registered after MapControllers()	Register header middleware before MapControllers()

15.5. Secrets Management and Secure Configuration

Builds on: 12.4 (configuration -- secrets are a config source), 12.7 (environments -- secrets differ per environment).

The four-tier secrets strategy

```
Tier 1 -- Local development:
  dotnet user-secrets
  Stored outside project directory
  Per-developer, never committed to Git

Tier 2 -- CI/CD pipelines:
  GitHub Actions secrets, Azure DevOps vars
  Environment variables set at build time

Tier 3 -- Staging/Production (self-hosted):
  Environment variables on the server
  Docker env vars, Kubernetes secrets

Tier 4 -- Production (cloud/enterprise):
  Azure Key Vault with Managed Identity
  No credentials in config at all
```

Tier 1 -- dotnet user-secrets

```
# Initialise:
dotnet user-secrets init --project src/MyApp.Api

# Set secrets:
dotnet user-secrets set\
    "JwtSettings:Secret"\
    "dev-secret-min-32-chars-xyz"\
    --project src/MyApp.Api

dotnet user-secrets set\
    "ConnectionStrings:DefaultConnection"\
    "Server=localhost;Database=MyAppDev;"\
    --project src/MyApp.Api

# List / remove / clear:
dotnet user-secrets list --project src/MyApp.Api
dotnet user-secrets remove "JwtSettings:Secret"
dotnet user-secrets clear

# Stored OUTSIDE project directory:
# Win: %APPDATA%\Microsoft\UserSecrets\<guid>\
#      secrets.json
# Linux: ~/.microsoft/usersecrets/<guid>/
#        secrets.json
```

Tier 2/3 -- Environment variables

```
# Double underscore = section separator:
export JwtSettings__Secret="prod-secret-key"
export ConnectionStrings__DefaultConnection=\
    "Server=prod-db;Database=MyApp;"

# Docker:
docker run\
  -e "JwtSettings__Secret=prod-secret"\
  -e "ConnectionStrings__DefaultConnection=..."\
```

```
  myapp:latest

# Kubernetes:
# apiVersion: v1
# kind: Secret
# stringData:
#   JwtSettings__Secret: prod-secret-key
```

Tier 4 -- Azure Key Vault

```
// dotnet add package
//   Azure.Extensions.AspNetCore.Configuration.Secrets
// dotnet add package Azure.Identity

if (!builder.Environment.IsDevelopment())
{
    var kvUri = new Uri(
        builder.Configuration["KeyVaultUri"]!);
    builder.Configuration.AddAzureKeyVault(
        kvUri, new DefaultAzureCredential());
    // Managed identity -- no credentials needed!
}

// Key Vault naming: use -- not : or __
// appsettings: JwtSettings:Secret
// Key Vault:   JwtSettings--Secret
```

Validating secrets at startup

```
// Fail fast if required secret is missing:
var required = new[]
{
    "ConnectionStrings:DefaultConnection",
    "JwtSettings:Secret"
};
foreach (var key in required)
{
    if (string.IsNullOrWhiteSpace(
            builder.Configuration[key]))
        throw new InvalidOperationException(
            $"Required secret '{key}' missing."
            + " Set via user-secrets (dev) or "
            + "environment variables (prod).");
}
```

Key Takeaways for 15.5

- Never commit secrets to source control -- JWT keys, passwords, API keys
- Tier 1 (Dev): dotnet user-secrets -- stored outside project, per-developer
- Tier 2 (CI/CD): env vars in pipeline -- __ as section separator
- Tier 3 (Server): Docker env vars / Kubernetes secrets
- Tier 4 (Cloud): Azure Key Vault with managed identity -- no credentials in config
- Validate required secrets at startup -- fail fast with clear message

```
Priority (highest first):
  Environment variables
  User secrets (Development only)
  appsettings.{Environment}.json
  appsettings.json
```

Debugging Tips

Symptom	Cause	Fix
User secrets not loading in Development	ASPNETCORE_ENVIRONMENT not set to Development	Check launchSettings.json; set env var explicitly
Azure Key Vault secrets not found	Secret name uses : instead of --	Key Vault names use --: JwtSettings--Secret
DefaultAzureCredential fails locally	Not logged into Azure CLI	Run az login; or set AZURE_CLIENT_ID/TENANT_ID/SECRET env vars
Env variable not overriding appsettings	Using : instead of __ as separator	Use double underscore: JwtSettings__Secret

15.6. Input Validation and SQL Injection Prevention

Builds on: 13.4 (model binding and validation -- data annotations), 14.6 (EF Core querying -- parameterised queries prevent SQL injection).

Three layers of validation

```
Layer 1 -- HTTP layer:
  Data annotations, FluentValidation
  Runs before action method
  Returns 400 Bad Request automatically

Layer 2 -- Business logic layer (service):
  Domain rules beyond format validation
  'Amount must not exceed credit limit'
  Throws domain exceptions

Layer 3 -- Database layer (EF Core):
  Constraints, NOT NULL, UNIQUE
  Last resort -- should not be first to catch
```

Data annotations and custom validators

```
public class CreateOrderDto
{
    [Required]
    [StringLength(100, MinimumLength = 2)]
    public string CustomerName { get; set; }
        = string.Empty;

    [Required]
    [EmailAddress]
    [MaxLength(200)]
```

```
    public string CustomerEmail { get; set; }
        = string.Empty;

    [Range(0.01, 1_000_000)]
    public decimal Amount { get; set; }

    [RegularExpression(@"^[A-Z]{2}\d{6}$")]
    public string? Reference { get; set; }
}

// Custom attribute:
public class FutureDateAttribute
    : ValidationAttribute
{
    protected override ValidationResult? IsValid(
        object? value, ValidationContext ctx)
    {
        if (value is DateTime date
            && date <= DateTime.UtcNow)
            return new ValidationResult(
                "Date must be in the future",
                [ctx.MemberName!]);
        return ValidationResult.Success;
    }
}
```

SQL injection prevention -- EF Core

```
// SAFE -- EF Core parameterises LINQ automatically:
var evil = "'; DROP TABLE Users; --";
var user = await _context.Users
    .Where(u => u.Email == evil)
    .FirstOrDefaultAsync(ct);
// SQL: WHERE Email = @p0  <- safe parameter

// SAFE -- FromSqlInterpolated:
var orders = await _context.Orders
    .FromSqlInterpolated(
        $"SELECT * FROM Orders WHERE Amount>{min}")
    .ToListAsync(ct);

// DANGEROUS -- string concatenation:
var orders = await _context.Orders
    .FromSqlRaw(
        "SELECT * FROM Orders WHERE Status = '"
        + status + "'")  // INJECTION RISK!
    .ToListAsync(ct);

// SAFE -- FromSqlRaw with parameters:
var orders = await _context.Orders
    .FromSqlRaw(
        "SELECT * FROM Orders WHERE Status={0}",
        status)
    .ToListAsync(ct);
```

Mass assignment and path traversal

```
// Mass assignment -- always use DTOs:
// ■ DANGEROUS:
public IActionResult Create(Order order) { }
// Attacker sets IsAdmin, Price, UserId etc.

// ■ SAFE:
public IActionResult Create(CreateOrderDto dto)
{
    var order = new Order
    {
        CustomerName = dto.CustomerName,
        Amount       = dto.Amount,
        UserId       = User.GetUserId(), // server
        CreatedAt    = DateTime.UtcNow,  // server
        Status       = "Pending"         // server
    };
}

// Path traversal -- validate file paths:
var safeFile = Path.GetFileName(fileName);
var uploadDir = Path.GetFullPath(_uploadDir);
var fullPath  = Path.GetFullPath(
    Path.Combine(uploadDir, safeFile));

if (!fullPath.StartsWith(uploadDir,
        StringComparison.OrdinalIgnoreCase))
    return BadRequest("Invalid file path");
```

Key Takeaways for 15.6

- Three layers: HTTP (annotations) -> Service (business rules) -> DB (constraints)
- EF Core parameterises LINQ automatically -- SQL injection through LINQ impossible
- Never use FromSqlRaw with string concatenation -- always use {0} parameters
- Mass assignment: always use DTOs; never bind request body to domain entity
- Path traversal: GetFileName + GetFullPath + verify stays inside allowed directory

Debugging Tips

Symptom	Cause	Fix
FluentValidation not running	Validators not registered or not wired	AddValidatorsFromAssemblyContaining<T>() + AddFluentValidationAutoValidation()
FromSqlRaw returning no results despite data	String concatenation used instead of {0} parameter	Replace concat with {0} placeholder or use FromSqlInterpolated
Mass assignment overwriting server fields	Binding entity type directly instead of DTO	Change action parameter to DTO; map manually to entity
400 but no error detail in response	[ApiController] returns ValidationProblemDetails	Read response.errors object in the 400 body

15.7. Data Protection API and Encryption

Builds on: 15.5 (secrets -- encryption keys are secrets), 12.2 (DI -- Data Protection registered on builder.Services).

Setting up Data Protection

```
// Dev -- keys in memory (lost on restart):
builder.Services.AddDataProtection();

// Production -- persist keys to disk:
builder.Services.AddDataProtection()
    .PersistKeysToFileSystem(
        new DirectoryInfo("/var/keys/myapp"))
    .SetApplicationName("MyApp")
    .SetDefaultKeyLifetime(
        TimeSpan.FromDays(90));

// Production -- Azure Blob + Key Vault:
builder.Services.AddDataProtection()
    .PersistKeysToAzureBlobStorage(
        new Uri("https://storage.blob.core"
            + ".windows.net/keys/keys.xml"),
        new DefaultAzureCredential())
    .ProtectKeysWithAzureKeyVault(
        new Uri("https://vault.azure.net/"
            + "keys/myapp-key"),
        new DefaultAzureCredential())
    .SetApplicationName("MyApp");
```

IDataProtector -- encrypt and decrypt

```
public class TokenService
{
    private readonly IDataProtector _protector;

    public TokenService(
        IDataProtectionProvider provider)
        => _protector = provider
            .CreateProtector(
                "MyApp.TokenService.v1");

    public string Protect(string plaintext)
        => _protector.Protect(plaintext);

    public string Unprotect(string ciphertext)
        => _protector.Unprotect(ciphertext);

    public string? TryUnprotect(string cipher)
    {
        try { return _protector.Unprotect(cipher); }
        catch (CryptographicException) { return null; }
    }
}
```

```
// Purpose string isolation:
// Email and password reset protectors have
// different purposes -- cannot decrypt each other
// Prevents token reuse attacks
```

ITimeLimitedDataProtector -- expiring tokens

```
public class EmailConfirmationService
{
    private readonly ITimeLimitedDataProtector
        _protector;

    public EmailConfirmationService(
        IDataProtectionProvider provider)
        => _protector = provider
            .CreateProtector(
                "MyApp.EmailConfirmation.v1")
            .ToTimeLimitedDataProtector();

    // Token expires in 24 hours:
    public string GenerateToken(string email)
        => _protector.Protect(
            email,
            lifetime: TimeSpan.FromHours(24));

    // Returns null if expired or tampered:
    public string? ValidateToken(string token)
    {
        try
        {
            return _protector.Unprotect(
                token, out var expiry);
        }
        catch (CryptographicException)
        {
            return null;
        }
    }
}
```

Hashing vs Encryption

```
// Encryption (two-way) -- use when you need
// to READ the original value:
// PII (tax ID, bank account), tokens, API keys

// Hashing (one-way) -- use when you only
// need to VERIFY the value:
// Passwords, API key verification

// ■ WRONG -- encrypting passwords:
customer.PasswordHash =
    _protector.Protect(password);

// ■ CORRECT -- hashing passwords:
```

```
// Identity does this automatically.
// Or use BCrypt directly:
// dotnet add package BCrypt.Net-Next
var hash = BCrypt.Net.BCrypt
    .HashPassword(password);
var valid = BCrypt.Net.BCrypt
    .Verify(password, hash);
```

Key Takeaways for 15.7

- Data Protection API = ASP.NET Core built-in authenticated encryption
- AddDataProtection() -- persist keys to file/Redis/Azure Blob in production
- CreateProtector(purpose) -- isolated encryptor; different purposes cannot decrypt each other
- ITimeLimitedDataProtector -- tokens that automatically expire (email confirm, pwd reset)
- Keys auto-rotate every 90 days; old keys kept for decryption
- Encryption (two-way) for data you need to read; Hashing (one-way) for passwords

```
Scenario                  | Tool
--------------------------|---------------------------
Email confirmation token  | ITimeLimitedDataProtector
Password reset link       | ITimeLimitedDataProtector
PII at rest (tax ID)      | IDataProtector + converter
Passwords                 | BCrypt / Identity (hash)
```

Debugging Tips

Symptom	Cause	Fix
CryptographicException on restart in Dev	Keys stored in memory -- lost on restart	Use PersistKeysToFileSystem() even in Development
Token valid on one server but not another	Each server has its own key ring	Use shared storage: Azure Blob, Redis, or shared file path
CryptographicException after redeployment	Keys not persisted between deployments	Persist keys outside app directory -- not in bin/ or publish/
Encrypted column too short in database	Encrypted value is longer than plaintext	Set HasMaxLength(500) or higher for encrypted string columns

15.8. Audit Logging and Sensitive Data Handling

Builds on: 14.2 (DbContext -- audit hooks into SaveChangesAsync), 12.5 (ILogger -- audit events written through logging), 15.7 (Data Protection -- sensitive fields encrypted before logging).

What to audit

```
ALWAYS audit:
  Authentication (login, logout, failed login)
  Authorisation failures (403)
  Data creation, modification, deletion
  Data export (who downloaded what)
```

```
  Permission changes (role assignments)

DO NOT audit:
  High-frequency reads (every GET)
  Health check endpoints
  Static file serving
```

Audit log entity

```
public class AuditLog
{
    public long Id { get; set; }

    // Who:
    public int? UserId { get; set; }
    public string UserEmail { get; set; }
        = string.Empty;

    // What:
    public string Action { get; set; }
        = string.Empty;  // Created/Updated/Deleted
    public string EntityType { get; set; }
        = string.Empty;  // Order, Customer
    public string EntityId { get; set; }
        = string.Empty;
    public string? OldValues { get; set; }  // JSON
    public string? NewValues { get; set; }  // JSON

    // Context:
    public string? IpAddress { get; set; }
    public string? CorrelationId { get; set; }

    // When:
    public DateTime Timestamp { get; set; }
        = DateTime.UtcNow;
}
```

Automatic audit via SaveChangesAsync override

```
public override async Task<int> SaveChangesAsync(
    CancellationToken ct = default)
{
    var audits = new List<AuditLog>();
    var ip = _httpContext.HttpContext
        ?.Connection.RemoteIpAddress?.ToString();

    foreach (var entry in ChangeTracker.Entries()
        .Where(e =>
            e.Entity is not AuditLog &&
            e.State is
                EntityState.Added or
                EntityState.Modified or
                EntityState.Deleted))
    {
        var old = new Dictionary<string,object?>();
```

```
var nw  = new Dictionary<string,object?>();

foreach (var prop in entry.Properties)
{
    // Skip sensitive fields:
    var sensitive = prop.Metadata
        .PropertyInfo?
        .GetCustomAttribute<
            SensitiveDataAttribute>()
        is not null;

    if (sensitive)
    {
        if (prop.IsModified)
            nw[prop.Metadata.Name] =
                "[REDACTED]";
        continue;
    }

    if (entry.State == EntityState.Added)
        nw[prop.Metadata.Name] =
            prop.CurrentValue;
    else if (entry.State ==
            EntityState.Deleted)
        old[prop.Metadata.Name] =
            prop.OriginalValue;
    else if (prop.IsModified)
    {
        old[prop.Metadata.Name] =
            prop.OriginalValue;
        nw[prop.Metadata.Name] =
            prop.CurrentValue;
    }
}

audits.Add(new AuditLog
{
    UserId     = _currentUser.UserId,
    UserEmail  = _currentUser.Email
        ?? string.Empty,
    Action     = entry.State switch {
        EntityState.Added    => "Created",
        EntityState.Modified => "Updated",
        EntityState.Deleted  => "Deleted",
        _                    => "Unknown"
    },
    EntityType = entry.Entity
        .GetType().Name,
    OldValues  = old.Count > 0
        ? JsonSerializer.Serialize(old)
        : null,
    NewValues  = nw.Count > 0
        ? JsonSerializer.Serialize(nw)
        : null,
    IpAddress  = ip,
```

```
            Timestamp  = DateTime.UtcNow
        });
    }

    var result = await base.SaveChangesAsync(ct);

    if (audits.Any())
    {
        AuditLogs.AddRange(audits);
        await base.SaveChangesAsync(ct);
    }

    return result;
}
```

Auth event audit (outside EF Core)

```
// Login success/failure audited in controller:
await _audit.LogAsync(new AuditLog
{
    Action     = "LoginFailed",
    EntityType = "Auth",
    UserEmail  = dto.Email,
    IpAddress  = ip,
    NewValues  = JsonSerializer.Serialize(
        new { Reason = "InvalidPassword" }),
    Timestamp  = DateTime.UtcNow
}, ct);

// Always return same message:
return Unauthorized(
    "Invalid email or password");
// Don't reveal whether email exists or not
```

Key Takeaways for 15.8

- Audit logs = who did what, to which resource, when, from where
- Override SaveChangesAsync to capture entity changes automatically
- Capture: UserId, UserEmail, Action, EntityType, EntityId, OldValues, NewValues, IP, Timestamp
- [SensitiveData] attribute on properties -- log [REDACTED] instead of actual value
- ICurrentUserService + IHttpContextAccessor -- access current user from DbContext
- Audit auth events in controller -- login success/failure outside EF Core
- Never delete audit records -- append-only; store in separate table or database

```
Who:   UserId, UserEmail
What:  Action, EntityType, EntityId,
       OldValues (JSON), NewValues (JSON)
Where: IpAddress, CorrelationId
When:  Timestamp (UTC)
```

Debugging Tips

Symptom	Cause	Fix
Audit log missing changes	SaveChangesAsync called on base class bypassing override	Ensure all saves go through the overriding context
EntityId empty in audit log	Generated ID not yet assigned at capture time	Capture ID after first SaveChangesAsync using entry.CurrentValues
Audit log flooding with reads	Read operations being audited	Only audit Added, Modified, Deleted states -- not Unchanged
Sensitive data in audit logs	[SensitiveData] attribute not checked during iteration	Check prop.Metadata.PropertyInfo?.GetCustomAttribute<SensitiveDataAttribute>()

Section 15 -- Complete Key Takeaways

```
Topic              | Core concept          | Key API
-------------------+-----------------------+------------------
15.1 JWT           | Stateless signed token| AddJwtBearer
15.2 Identity      | User store management | UserManager<T>
15.3 Authorization | Roles + policies      | [Authorize]
15.4 HTTPS/CORS    | Transport security    | UseHsts/UseCors
15.5 Secrets       | Four-tier strategy    | user-secrets
15.6 Validation    | Three-layer defence   | Annotations/EF
15.7 Data Protect  | Encrypt/decrypt data  | IDataProtector
15.8 Audit Logging | Who did what when     | SaveChanges hook
```

Section 15 complete. All 8 topics covered. Say next to continue to Section 16: API Communication (16.1 HttpClient and IHttpClientFactory, 16.2 CORS configuration).